

Datenbanken

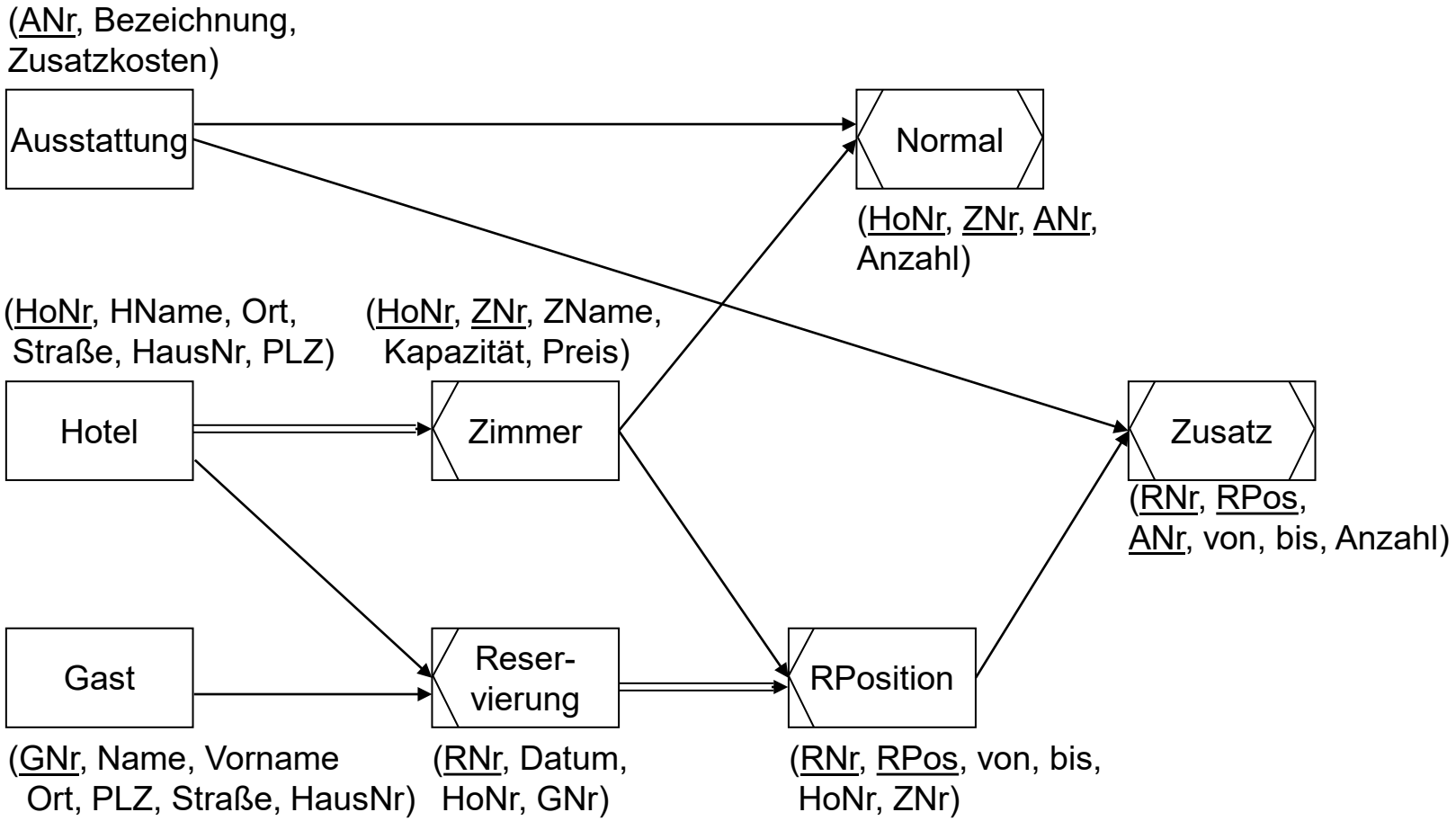
Prof. Dr.
Frank Kammer

Lösungen der Aufgaben im 5. Kapitel 3. Teil

TECHNISCHE HOCHSCHULE MITTELHESSEN
THM
CAMPUS GIESSEN
MNI
Mathematik, Naturwissenschaften und Informatik

5 Aufgaben – Stored Procedures

- Prozedur zum Anlegen eines Zimmers mit allen Attributen über den Hotelnamen



5 Lösung Zimmer anlegen – PostgreSQL

- Prozedur zum Anlegen eines Zimmers mit allen Attributen über den Hotelnamen

```
CREATE PROCEDURE z_einfuegen (ehname CHAR(50),
                              eznr INTEGER,
                              ezname CHAR(50),
                              ekapazitaet INTEGER,
                              epreis NUMERIC(7,2))
LANGUAGE plpgsql AS $$
DECLARE ehnr INTEGER;
BEGIN
  SELECT honr INTO ehnr FROM hotel WHERE hname = ehname;
  INSERT INTO zimmer (honr, znr, zname, kapazitaet, preis)
         VALUES(ehnr, eznr, ezname, ekapazitaet, epreis);
END; $$

CALL z_einfuegen('Hotel Adlon', 100, 'Zweibrettzimmer', 2, 7.90);
```

5 Lösung Zimmer anlegen – MySQL

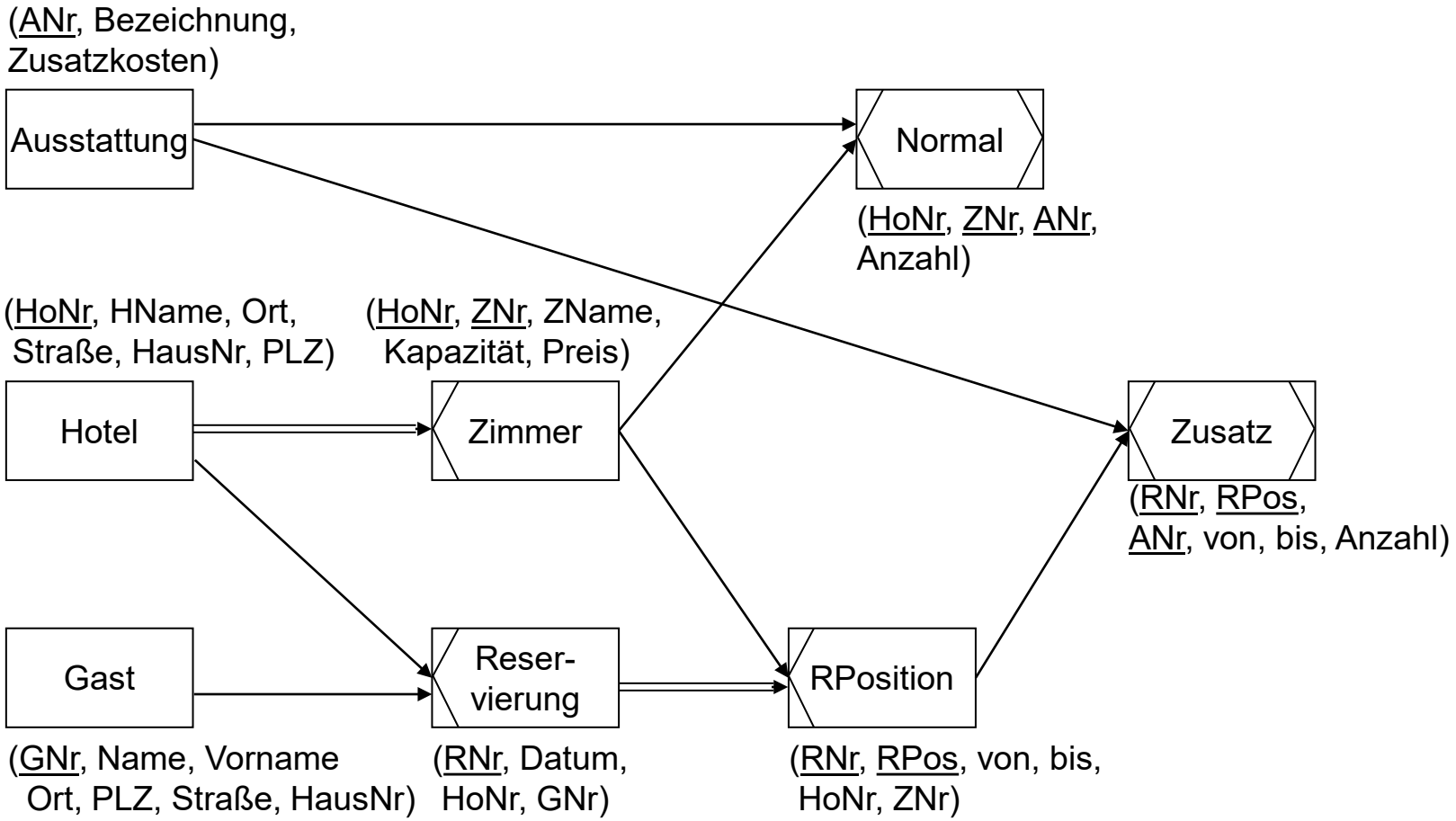
- Prozedur zum Anlegen eines Zimmers mit allen Attributen über den Hotelnamen

```
DELIMITER //
CREATE PROCEDURE z_einfuegen (ehname CHAR(50),
                              eznr INTEGER,
                              ezname CHAR(50),
                              ekapazitaet INTEGER,
                              epreis NUMERIC(7,2))
BEGIN
  DECLARE ehnr INTEGER;
  SELECT honr INTO ehnr FROM hotel WHERE hname = ehname;
  INSERT INTO zimmer (honr, znr, zname, kapazitaet, preis)
         VALUES(ehnr, eznr, ezname, ekapazitaet, epreis);
END //
DELIMITER ;

CALL z_einfuegen('Hotel Adlon', 100, 'Fürstensuite', 6, 987.90);
```

5 Aufgaben – Stored Procedures

- Prozedur zum Anpassen der Mehrwertsteuer bei Zimmer- und Ausstattungspreisen. Eingabe ist alte und neue MwSt.



5 Lösung Mehrwertsteuer anpassen – PostgreSQL

- Prozedur zum Anpassen der Mehrwertsteuer bei Zimmer- und Ausstattungspreisen. Eingabe ist alte und neue MwSt.

```
CREATE PROCEDURE mwst_anpassen (
                    neu NUMERIC(4,2), alt NUMERIC(4,2))
LANGUAGE plpgsql AS $$
BEGIN

  UPDATE zimmer
  SET preis = preis / (100.0 + alt) * (100.0 + neu);

  UPDATE ausstattung
  SET zusatzkosten = zusatzkosten / (100.0 + alt) * (100.0 + neu);

END; $$

CALL mwst_anpassen (16.0, 19.0); -- 1.7.2020
CALL mwst_anpassen (19.0, 16.0); -- 1.1.2021
```

5 Lösung Mehrwertsteuer anpassen – MySQL

- Prozedur zum Anpassen der Mehrwertsteuer bei Zimmer- und Ausstattungspreisen. Eingabe ist alte und neue MwSt.

```
DELIMITER //
CREATE PROCEDURE mwst_anpassen (
                    neu NUMERIC(4,2), alt NUMERIC(4,2))
BEGIN

  UPDATE zimmer
  SET preis = preis / (100.0 + alt) * (100.0 + neu);

  UPDATE ausstattung
  SET zusatzkosten = zusatzkosten / (100.0 + alt) * (100.0 + neu);

END //
DELIMITER ;

CALL mwst_anpassen (16.0, 19.0); -- 1.7.2020
CALL mwst_anpassen (19.0, 16.0); -- 1.1.2021
```

5 Aufgaben – Stored Procedures

- Prozedur zur Erstellung einer Tabelle, in der eine Maschine linear abgeschrieben wird
  Beispiel:
  - Wert 21.000 Euro
  - Dauer: 7 Jahre

  $\Rightarrow$ Rate: 21.000 Euro / 7 Jahre = 3.000 Euro/Jahr

Jahr	Rate	Restwert
0	0	21.000
1	3.000	18.000
2	3.000	15.000
3	3.000	12.000
4	3.000	9.000
5	3.000	6.000
6	3.000	3.000
7	3.000	0

5 Lösung Lineare Abschreibung – PostgreSQL

```
CREATE PROCEDURE linear (IN betrag NUMERIC(10,2), dauer INTEGER)
LANGUAGE plpgsql AS $$
DECLARE i INTEGER;
DECLARE rbwert NUMERIC(10,2);
DECLARE abwert NUMERIC(10,2);
BEGIN
 abwert := betrag /dauer ;
 rbwert := betrag;
 INSERT INTO abschr (jahr, abschreibung, restbuchwert)
             VALUES (0, 0.00, betrag);
 i := 1;
 WHILE rbwert>0 LOOP
   rbwert := rbwert - abwert;
   INSERT INTO abschr (Jahr, Abschreibung, Restbuchwert)
          VALUES (i, abwert, rbwert);
   i := i+1;
 END loop;
END; $$
```

```
CREATE TABLE abschr (
    Jahr INTEGER,
    abschreibung DECIMAL(10 , 2 ),
    restbuchwert DECIMAL(10 , 2 )
);
```

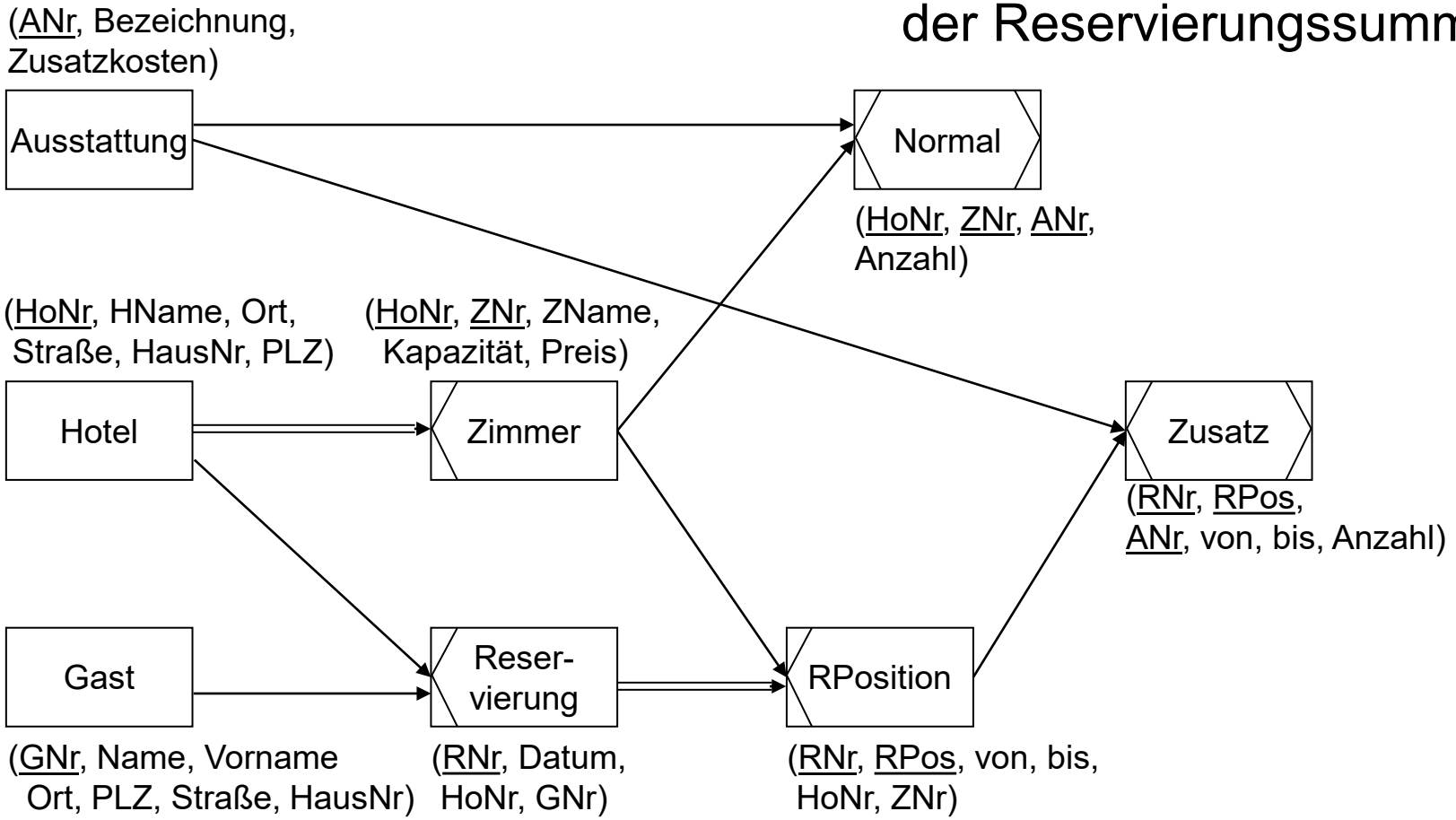
5 Lösung Lineare Abschreibung – MySQL

```
DELIMITER //
CREATE PROCEDURE linear (IN betrag NUMERIC(10,2), dauer INTEGER)
BEGIN
  DECLARE i INTEGER;
  DECLARE rbwert NUMERIC(10,2);
  DECLARE abwert NUMERIC(10,2);
  SET abwert = betrag/dauer;
  SET rbwert = betrag;
  INSERT INTO abschr (jahr, abschreibung, restbuchwert)
                     VALUES (0, 0.00, betrag);
  set i = 1;
  WHILE rbwert>0 DO
    SET rbwert = rbwert - abwert;
    INSERT INTO abschr (Jahr, Abschreibung, Restbuchwert)
                       VALUES (i, abwert, rbwert);
    SET i = i+1;
END WHILE;
END//
DELIMITER ;
```

```
CREATE TABLE abschr (
    Jahr INTEGER,
    abschreibung DECIMAL(10 , 2 ),
    restbuchwert DECIMAL(10 , 2 )
);
```

5 Aufgaben – Trigger

- Verwaltung einer Reservierungssumme in Reservierung nach jeder neuen Rechnungsposition. Vorab Erstellung/Initialisierung der Reservierungssumme.



5 Lösung Reservierungssumme – PostgreSQL

```
ALTER TABLE reservierung ADD COLUMN rsumme NUMERIC(8,2) DEFAULT 0;
UPDATE reservierung SET rsumme =
    (SELECT SUM((rp.bis-rp.von)*z.preis)
     FROM rposition rp JOIN zimmer z USING (znr, honr)
     WHERE rp.rnr = reservierung.rnr
     GROUP BY rp.rnr);
```

```
CREATE FUNCTION ttsumme () RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE s NUMERIC(8,2) DEFAULT 0;
DECLARE p NUMERIC(8,2) DEFAULT 0;
BEGIN
  SELECT rsumme INTO s FROM reservierung r WHERE r.rnr = NEW.rnr;
  SELECT preis INTO p FROM zimmer
     WHERE zimmer.znr = NEW.znr AND zimmer.honr = NEW.honr;
  s := s + (NEW.bis-NEW.von)*p;
  UPDATE reservierung SET rsumme = s WHERE rnr = NEW.rnr;
  RETURN new;
END; $$

CREATE TRIGGER rechsumme BEFORE INSERT ON rposition
   FOR EACH ROW execute function ttsumme();
```

5 Lösung Reservierungssumme – MySQL

```
ALTER TABLE reservierung ADD COLUMN rsumme NUMERIC(8,2) DEFAULT 0;

UPDATE reservierung SET rsumme =
    (SELECT SUM((rp.bis-rp.von)*z.preis)
     FROM rposition rp JOIN zimmer z USING (znr, honr)
     WHERE rp.rnr = reservierung.rnr
     GROUP BY rp.rnr);
```

```
DELIMITER //
CREATE TRIGGER rechsumme FOR rposition AFTER INSERT
BEGIN
  DECLARE s NUMERIC(8,2) DEFAULT 0;
  DECLARE p NUMERIC(8,2) DEFAULT 0;
  SELECT rsumme INTO s FROM rechnung WHERE rnr = NEW.rnr;
  SELECT preis INTO p FROM zimmer
                      WHERE znr = NEW.znr AND honr = NEW.honr;
  s = s + (NEW.bis-NEW.von)*p;
  UPDATE hotel SET rsumme = s WHERE rnr = NEW.rnr;
END//
DELIMITER ;
```